

iOS Prototype for Guided Storytelling Analytics App

This Markdown file outlines the structure of an iOS prototype (iPadOS-friendly) for a guided storytelling app. The goal of the app is to help first-time parents understand their child's behaviour by logging daily activities, moods and reflections, then presenting an analytics dashboard.

1 Data Menu: Viewing and Creating Dummy Data

The first menu displays the data stored in the app. We will use **SwiftData** (introduced in iOS 17) for local persistence. Each record represents an activity or "story event". Fields:

- **childName** (String)
- **childAge** (Int)
- **childGender** (String)
- **scene** (enum: morning, afternoon, evening, night)
- **place** (enum: home, school, park, etc.)
- **activity** (String)
- **mood** (String or mood enum)
- **reason** (String – optional explanation from the child)
- **parentNote** (String – optional)
- **date** (Date)

We want at least seven days of dummy data. You can insert sample records with a helper function in `PreviewProvider` or inside an `onAppear` block:

```
@Model
final class StoryEvent {
    @Attribute(.unique) var id: UUID = UUID()
    var childName: String
    var childAge: Int
    var childGender: String
    var scene: Scene
    var place: Place
    var activity: String
    var mood: Mood
    var reason: String?
    var parentNote: String?
    var date: Date
}
```

Use an enum for scenes and mood:

```

enum Scene: String, Codable, CaseIterable {
    case morning, afternoon, evening, night
}

enum Mood: String, Codable, CaseIterable {
    case happy, sad, angry, tired, excited, scared
}

enum Place: String, Codable, CaseIterable {
    case home, school, park, other
}

```

In the **Data Menu**, display a list of entries by date. Provide a button to add new entries (for testing), but in a real app data would be created via the storytelling UI.

2 Dashboard Menu: Visualising Patterns

The dashboard shows aggregated patterns. Use **Charts** (SwiftUI Charts) to build simple bar and line charts. Possible analytics:

1. **Mood frequency by scene** – a bar chart showing how often each mood occurs in morning/afternoon/evening/night.
2. **Activity vs mood** – a heatmap or stacked bar showing the distribution of moods across activities.
3. **Place vs mood** – to see if certain locations correspond to happier or more negative moods.
4. **Trend over time** – a line chart showing the mood of each day over the 7-day period.

Example chart code:

```

Chart(data) { event in
    BarMark(
        x: .value("Scene", event.scene.rawValue),
        y: .value("Count", 1)
    )
    .foregroundColor(by: .value("Mood", event.mood.rawValue))
}

```

In addition to charts, provide an **insight summary**. This is a simple `Text` view that synthesises patterns found in the data. It might say:

"This week, Leo was often tired in the evening after playtime at the park. Morning moods were generally positive, especially during home activities."

The summary is not diagnostic; it is meant to help parents recognise triggers and discuss them with children. You can generate this text using simple rules: count occurrences and produce human-readable statements (no LLM needed).

3 Model Selection Menu: Choosing an Analysis Engine

The app provides a menu where parents can choose how the summary and charts are generated. Options might include:

- **Simple count & rules** (default) – uses basic Swift algorithms to count occurrences and produce insights. Appropriate for offline use and fast performance.
- **Core ML model** – uses a small classification or clustering model to detect patterns (e.g., mood prediction or clustering similar situations). You can train such a model offline (e.g. using Create ML) then embed it via the `.mlmodel` file.
- **LLM summarisation** – optionally, you could allow downloading a small language model (e.g. via Apple's forthcoming on-device model support). The user would choose a model from a list of local files. For privacy reasons, the app should process all data on device.

Downloading/Updating Models

Since iOS does not yet provide a unified UI for downloading third-party models, the app could integrate a **Model Gallery**. For example, Apple's *Core ML Model Gallery* or third-party repositories might be referenced via URL. Parents can select a model to download via a simple list, then the app uses `URLSession` to fetch the `.mlmodel` file and store it locally. Once downloaded, use `MLModel` to compile and load it.

```
func downloadModel(from url: URL) async throws -> URL {
    let (location, _) = try await URLSession.shared.download(from: url)
    let documents = FileManager.default.urls(for: .documentDirectory,
    in: .userDomainMask).first!
    let destURL = documents.appendingPathComponent(url.lastPathComponent)
    try? FileManager.default.removeItem(at: destURL)
    try FileManager.default.copyItem(at: location, to: destURL)
    return destURL
}
```

After downloading, use `MLModel.load(contentsOf:)` and integrate it into the analytics pipeline.

Why Core ML?

Core ML provides on-device inference with privacy and no network dependency. Apple's documentation emphasises that all inference can happen on device for user data security. A small classifier (e.g. decision tree or logistic regression) can run efficiently and does not require extensive memory. Use `Create ML` or `mltools` to train the model outside the app.

Feasibility and Gaps

- **Feasibility:** All components rely on native frameworks (SwiftUI, SwiftData, Charts, Core ML). Data is stored locally with SwiftData; charts are easy to build; model selection uses existing system APIs. Downloading and integrating an `.mlmodel` file requires asynchronous file handling but is documented in Apple's Core ML guides.
- **Gaps:** We do not have citations from Apple's documentation due to limited research time. In a real project, consult Apple's developer documentation for precise APIs (SwiftData usage, Charts

examples, Core ML model handling). Also, training an effective ML model requires a labelled dataset; the dummy data in this prototype will not generalise.

Next Steps

1. Implement the `StoryEvent` model and persist to `SwiftData`.
2. Create SwiftUI views:
3. `TableView` to list and add dummy events.
4. `DashboardView` for charts and summary.
5. `ModelSelectionView` for selecting analysis engine and downloading models.
6. Write simple analysis logic (counts & rules) and optionally integrate a Core ML model.
7. Test on iPhone first; later adjust layout for iPad.

By following this plan you can produce a proof-of-concept analytics menu for your storytelling app, even though the research did not provide direct Apple documentation citations. The concept remains feasible and aligns with the app's mission: to help parents reflect on their child's daily experiences.
